

Mechanisms of OthelloGPT

Zihang Wen, Can Rager

Summary

- We study the internal mechanisms of OthelloGPT, a small transformer trained on the board game Othello.
- We localize four critical mechanisms:
 - Just Flipped Computation
 - Cross-token board state copying
 - Flipped-board state computation
 - Valid move lookup
- We highlight these surprising findings:
 - ... (let's talk about what to put here)
 - A token position only contains currently flipped information, not previously flipped information. (We actually need evidence for this, but I think its)

Introduction

(One paragraph on the motivation: Understanding the most complex toy model to date would enable evaluation of interpretability agents.)

Understanding the internal algorithms of transformer models is a central goal of mechanistic interpretability. A long-term objective is to construct human-interpretable replacement models that explain model behavior in terms of explicit representations and computations, while remaining predictive of the original model's outputs and its behavior under interventions, and failure cases. While toy models such as Tracr \citep{lindner_tracr_2023} have provided valuable insights in controlled synthetic settings, their simplicity limits the development and evaluation of scalable techniques. To evaluate progress in (automated) interpretability techniques, we need benchmarks that are complex enough to reveal non-trivial structure, yet grounded enough to allow systematic analysis.

Othello-GPT—a transformer trained to predict valid moves in Othello—presents an ideal testbed for interpretability research, substantially more complex than existing toy models yet grounded in clear game logic that enables meaningful reverse-engineering. The game's clearly defined rules and world state provide ground truth that enables programmatic analysis and hypothesis verification about model components. Prior work has suggested that Othello-GPT operates through two computational phases: an initial "reconstruction" phase that transforms the input move sequence into an internal board state representation, followed by a "prediction" phase that

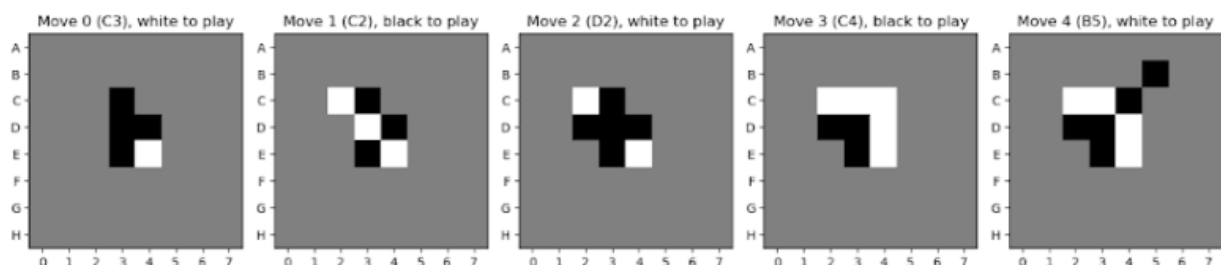
uses this board state to identify valid moves \citep{li_emergent, nanda_emergent_2023}. This positions Othello-GPT to potentially become the most complex model for which substantial mechanistic understanding has been achieved.

Preliminaries

Background on OthelloGPT

Othello is a 2-player game on an 8x8 board where players take turns placing pieces. Placing a piece that traps a line of opponent pieces between itself and an existing friendly piece flips those pieces. Only moves that flip at least one piece are legal. The game ends when the board is full, and the player with the most pieces wins.

Li et al. [1] proposed using the board game Othello as a testbed for neural network interpretability. They trained OthelloGPT, a small transformer with 25 million parameters to predict the next legal moves based on the move history. The transformer's vocabulary exclusively consists of playable tiles. An example input sequence describing 5 moves into the game is: [C3, C2, D2, C4, B5], where BLACK's moves are highlighted grey.



The transformer is trained to have all valid moves as top logits. In the example above, the valid moves for white are {D1, E1, E2, F2, F3, C5, B4}. The solution of next valid moves is not explicitly given in the input. Correct prediction of next valid moves requires internally keeping track of the current board state, which makes this task particularly useful for studying internal world models. Li et al. [1] found that it spontaneously learned an emergent world representation of the board. This internal representation could be recovered using non-linear probes, but not with linear probes. Through activation interventions, they demonstrated that this representation was causally linked to the model's predictions.

Building on this, Nanda et al. [2] and Hazineh et al. [3] showed that the board state is actually linearly encoded, but relative to the current player (MINE, YOURS) rather than absolute colors (BLACK, WHITE). These directions in activation space were also shown to be causally meaningful: intervening along them altered the model's behavior in predictable ways. Parsing

our previous examples as MINE and YOURS from the perspective of the player to make the next valid move:

[C3, C2, D2, C4, B5]

After the next move, the relative position changes:

[C3, C2, D2, C4, B5, B4]

Nanda et al. [2] further provided a mechanistic analysis of how the model detects EMPTY tiles and represents pieces FLIPPED in the most recent move.

Significant board state representations

Building on these findings, we choose the following variables in our work. We find the linear directions encoding these variables with linear probes.

- MINE = “mine” - “yours” probe directions
- BLANK = “blank” - (“mine” + “yours”) / 2 probe directions
- FLIPPED = “just flipped” - “not just flipped” probe directions
- PLAYED = “just played” - “not just played” probe directions

Note, these are not generally unit normalized, unless specified, or used in cosine similarity. We use “either mine or yours” as the counterfactual for blank.

MINE-, YOURS-, and Uniform-Heads

Nanda et al. [2] found that most attention heads either exclusively attend to MINE moves or YOURS moves. Jmaar [7] further quantifies that these patterns are approximately static across games, and pay more attention to recent moves. In line with prior work, we categorize these heads into three functional groups based on their attention patterns:

- Mine-Heads: The average pattern score referred to my moves is twice as large as the average pattern score referred to your moves.
- Yours-Heads: The average pattern score referred to your moves is twice as large as the average pattern score referred to my moves.
- Uniform-Heads: Else (Attention weights are evenly distributed across moves.)

Note: this threshold “two times” is quite arbitrary here; the difference between the means could be seen in Fig 5.

[Footnote: We mask the attention scores first column, corresponding to the first move as the key value, to filter out attention sink behavior. The model is not trained with a BOS token, so the attention sink coincides with the first move. For clean separation we recommend future OthelloGPT training runs to train with a BOS token, to cleanly separate attention sink behavior from attention patterns to my / your moves.]

Fig 4 shows the mean attention pattern

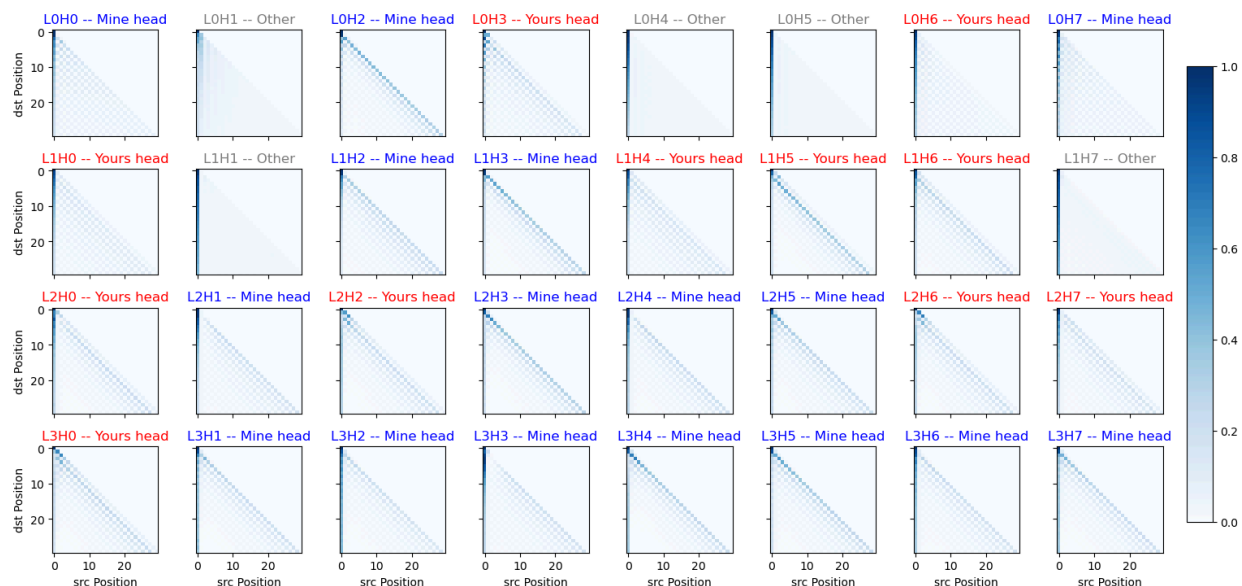


Fig 4: Mean attention pattern of each attention head across 500 games each.

Mean and standard deviation for all my moves (blue) and all your moves (blue) for each head across 500 games further quantify categorization into Mine heads and Yours heads. And for all Mine heads (blue), their pattern scores for your moves have both small mean and standard deviation. Similar for Yours heads.

Attention Pattern Standard Deviation Across Games

L0H0: $\mu=0.085$ $\sigma=0.021$ $\mu=0.002$ $\sigma=0.000$	L0H1: $\mu=0.051$ $\sigma=0.036$ $\mu=0.050$ $\sigma=0.037$	L0H2: $\mu=0.107$ $\sigma=0.020$ $\mu=0.002$ $\sigma=0.000$	L0H3: $\mu=0.002$ $\sigma=0.000$ $\mu=0.094$ $\sigma=0.021$	L0H4: $\mu=0.028$ $\sigma=0.018$ $\mu=0.029$ $\sigma=0.019$	L0H5: $\mu=0.027$ $\sigma=0.020$ $\mu=0.028$ $\sigma=0.021$	L0H6: $\mu=0.003$ $\sigma=0.001$ $\mu=0.057$ $\sigma=0.021$	L0H7: $\mu=0.071$ $\sigma=0.031$ $\mu=0.008$ $\sigma=0.003$
L1H0: $\mu=0.002$ $\sigma=0.002$ $\mu=0.075$ $\sigma=0.057$	L1H1: $\mu=0.025$ $\sigma=0.022$ $\mu=0.020$ $\sigma=0.014$	L1H2: $\mu=0.099$ $\sigma=0.042$ $\mu=0.002$ $\sigma=0.001$	L1H3: $\mu=0.104$ $\sigma=0.039$ $\mu=0.002$ $\sigma=0.001$	L1H4: $\mu=0.002$ $\sigma=0.002$ $\mu=0.082$ $\sigma=0.058$	L1H5: $\mu=0.002$ $\sigma=0.001$ $\mu=0.099$ $\sigma=0.047$	L1H6: $\mu=0.002$ $\sigma=0.002$ $\mu=0.087$ $\sigma=0.055$	L1H7: $\mu=0.027$ $\sigma=0.016$ $\mu=0.015$ $\sigma=0.009$
L2H0: $\mu=0.003$ $\sigma=0.002$ $\mu=0.073$ $\sigma=0.045$	L2H1: $\mu=0.076$ $\sigma=0.041$ $\mu=0.002$ $\sigma=0.001$	L2H2: $\mu=0.003$ $\sigma=0.001$ $\mu=0.084$ $\sigma=0.048$	L2H3: $\mu=0.101$ $\sigma=0.044$ $\mu=0.003$ $\sigma=0.002$	L2H4: $\mu=0.076$ $\sigma=0.050$ $\mu=0.003$ $\sigma=0.002$	L2H5: $\mu=0.093$ $\sigma=0.051$ $\mu=0.003$ $\sigma=0.002$	L2H6: $\mu=0.003$ $\sigma=0.002$ $\mu=0.087$ $\sigma=0.047$	L2H7: $\mu=0.003$ $\sigma=0.002$ $\mu=0.073$ $\sigma=0.036$
L3H0: $\mu=0.005$ $\sigma=0.001$ $\mu=0.098$ $\sigma=0.032$	L3H1: $\mu=0.078$ $\sigma=0.057$ $\mu=0.003$ $\sigma=0.002$	L3H2: $\mu=0.066$ $\sigma=0.053$ $\mu=0.003$ $\sigma=0.002$	L3H3: $\mu=0.069$ $\sigma=0.036$ $\mu=0.003$ $\sigma=0.001$	L3H4: $\mu=0.091$ $\sigma=0.056$ $\mu=0.003$ $\sigma=0.002$	L3H5: $\mu=0.102$ $\sigma=0.039$ $\mu=0.004$ $\sigma=0.001$	L3H6: $\mu=0.073$ $\sigma=0.047$ $\mu=0.003$ $\sigma=0.002$	L3H7: $\mu=0.082$ $\sigma=0.061$ $\mu=0.003$ $\sigma=0.002$

Fig 5: Mean and standard deviation of the attention patterns for my moves (blue) and your moves (red) across 500 games, computed separately for each head.

Multiple works identified mechanisms for narrow behaviors, including rule-based neurons [], the previous color circuit [], and probe projections, as detailed in the related work section. This work provides a unified description of internal mechanisms in OthelloGPT.

The four mechanisms

(explanation of each state and, show plot, and explain and interpret the evidence.)

- Just Flipped Computation

- L0 MLP, L1 MLP, partial L2 MLP: **board state** → **just flipped**

- Cross-token Board State Copying

- L1-L4(7): Yours (mine) heads: **just played** → **board state y(m)**
- L3-L5: Yours (mine) heads: flip (copy) **board state**
- ***just flipped** not copied*

- Flipped → Board State computation

- L0-L4 MLPs: **just flipped** → **yours**
- L1-L4(7): Yours (mine) heads: **just flipped** → **yours (mine)**

- Valid move Lookup

- Decision rules: Learn decision trees to predict MLP activations
- L5-L7 MLPs: **board state** → **valid move**

Board state computation

Just Flipped Computation

In the game of Othello, placing a piece flips one or more opponent's pieces. The "Just Flipped" mechanism in OthelloGPT identifies the tiles altered by the most recent move. Our analysis localizes this computation to the MLP layers in Layers 0 through 3, which write the directions of these flipped tiles into the residual stream.

Single move inspection

In line with prior work, we train probes at every residual stream layer to predict which tiles have been flipped. At a given residual stream site, we train a probe for each tile to binary classify "just_flipped" or "not_just_flipped". For a single move in a single game, we project the output of each component (MLP, Multi-Headed-Attention) as well as the full residual stream vector onto the "just_flipped" directions of linear probes.

Fig 1 (left) shows board states of a game after moves 7 and 8. In move 8, C1 is played, and tiles C2, C3, and C4 are flipped. Fig 1(right) shows the dot product of each model component with the “just flipped” probe direction $\text{just_flipped_projection}_i = \text{mlp_out} @ \text{“just flipped” probe direction}$ for tile i . The middle and last rows show the “mid” and “post” residual streams after the attention layers and mlp layers, respectively. Blue color represents that the component has a positive contribution (or writing positively) to a specific square being flipped. Black circles mark “C2, C3, and C4” squares that are flipped in move 8 and have a relatively strong blue color on the board. This plot shows that MLPs from L0 to L4 actively write these “squares being flipped” to the residual stream. We reproduce this figure for other moves in Appendix X. The investigation of single moves indicates that L0–L4 MLPs are the main model components writing to the “just flipped” direction. The next section provides a quantitative evaluation of this hypothesis.

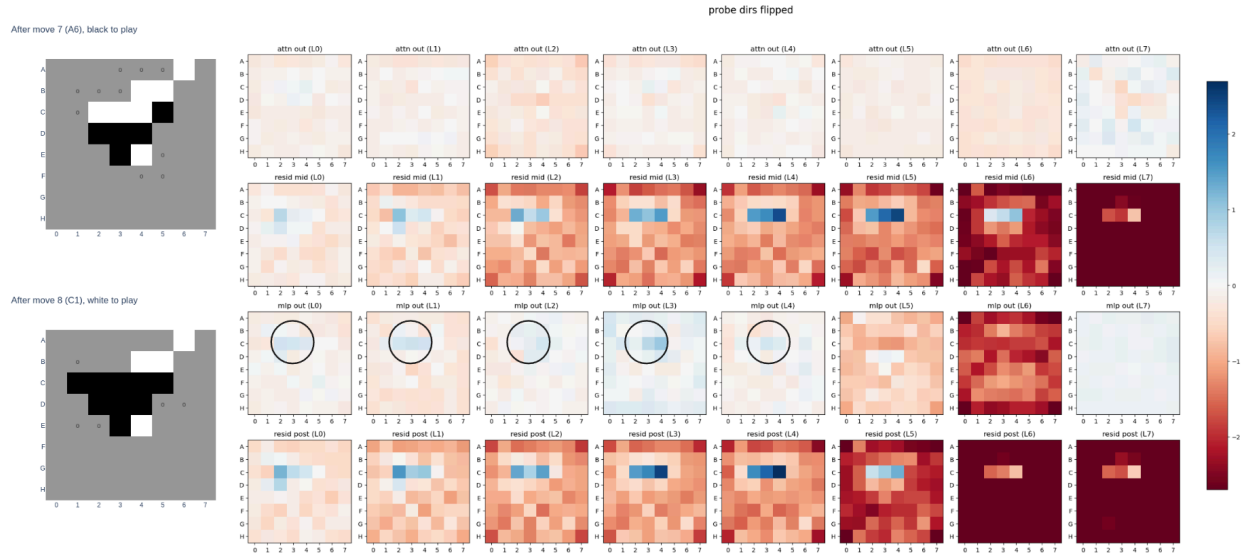


Fig 1: Left: Board states of a game after moves 7 and 8. Right: Heatmap of attributions of each component (attentions and mlps) on the “just flipped” probe direction after move 8. Black circled the tiles just flipped by move 8.

Quantitative evaluation across 500 games

Statistical validation across 500 games confirms that MLP L0 - L3 are the main model components writing to the “just flipped” probe direction.

For each move, we register the number of k flipped tiles as ground truth. For each model component, we compute the $\text{just_flipped_projection} = \text{component_out} @ \text{“just flipped” probe direction}$ scores for all tiles and select the top k tiles with the highest scores. We report the F1 score, treating the highest tile_projection as a predictor for the k ground truth flipped tiles.

The results per model component are shown in Fig 2. The x-axis is the move indices of each game (move 8 to move 30), and the y-axis is the F1 score, with color representing the number

of games that have this F1 score in each move. Each move index (column of a subplot) shows the distribution of F1 scores across 500 games. The distribution exhibits modes (quantized structure) because the average number of flipped tiles per move is low, leaving a few possible values for F1 scores. Model components MLP L0 - L3 stand out with high F1 classification scores.

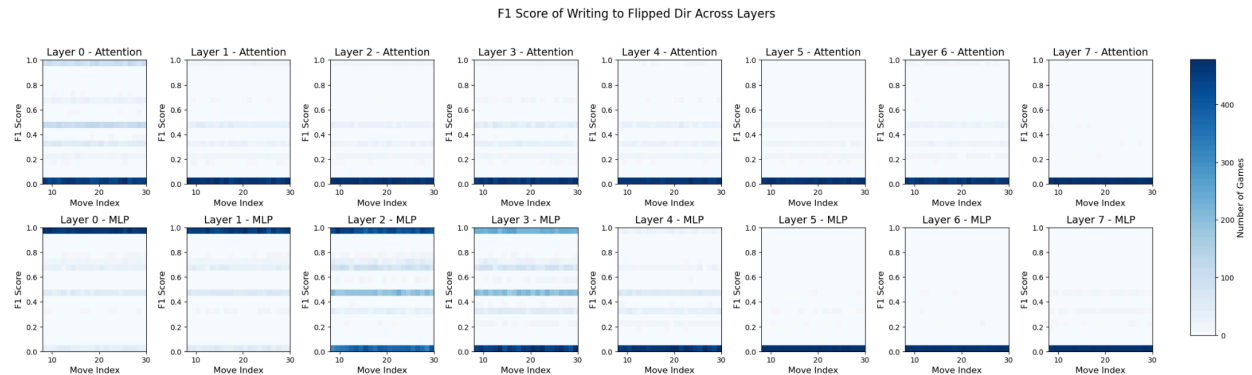
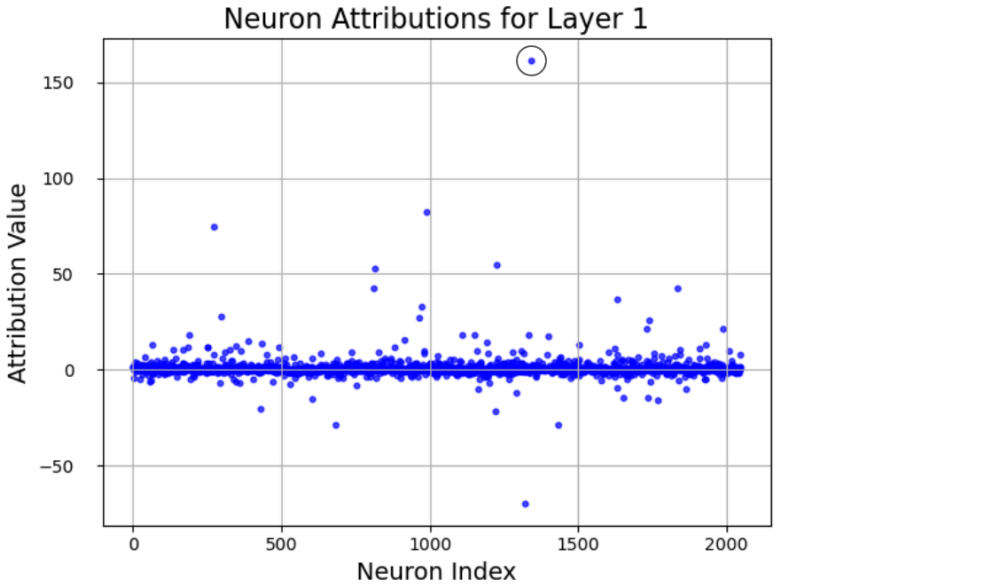


Fig 2: F1 score of top-k highest scores of $\text{tile_projection} = \text{mlp_out} @ \text{"just_flipped"}$ as a predictor of flipped tiles. k is the number of tiles that are actually flipped in each single move and board state. The x-axis is the move indices of each game (move 8 to move 30), and the y-axis is the F1 score, with color representing the number of games that have this F1 score in each move.

Individual MLP Neurons indicate "just_flipped"

We identified specific neurons driving this process. We first calculated each L1 neuron's attribution of "C2 being flipped" across all the moves with C2 flipped across 500 games in Fig 3 (top). We then show the heatmap of a neuron's input and output matrix projections on MINE, BLANK, FLIPPED, and PLAYED probes to show what it reads from and writes to. For instance, L1N1341 (highest value, circled in Fig 3 (top)) activates upon the play of a specific move (e.g., B1), then explicitly flips the tiles' board states and marks the tiles flipped (writes to flipped probe direction). In Appendix Fig S1, we also show the attribution value of layer 0 and give an example neuron L0N1578 which activates upon the play of a specific move (e.g., B1) and writes "just flipped" information for adjacent tiles.



(Rank 0: L1N1341) MLP Weights Projection for square C2

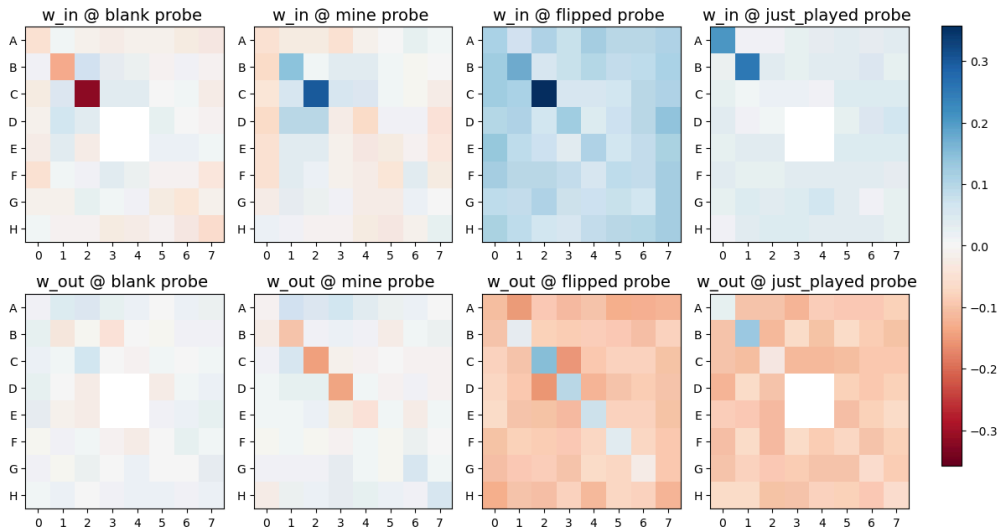


Fig 3: Top: Attribution score for each L1 neuron’s contribution to C2 flipped probe direction with all moves where C2 flipped over 500 games. Black circled neuron L1N1341. Bottom: cosine similarity of W_{in} and W_{out} with “blank”, “mine”, “just flipped”, “just played” probe directions, respectively.

JUST Flipped → Board State computation

We investigate which model components read “just flipped” and write “mine/yours” in similar fashion to the “just flipped” computation above. We found a distinct division of labor: while attention heads primarily manage information from previous turns, MLPs are the dominant contributors to updating the state of tiles flipped by the current move.

Qualitative inspection of single moves

Fig 10 (right) shows the projection of each component (MLP and Multi-Headed-Attention) onto the “mine” probe direction via `$component_out @ mine probe direction$`. The middle and last rows show the “mid” and “post” residual streams after the attention layers and mlp layers, respectively. Blue color represents that the component writes that a specific square is mine. Red color represents that the component writes that a specific square is yours. The red circles show the squares that are actually flipped by the most recent move (move 8). Fig 10 indicates that MLPs L0 - L4 write just flipped squares to yours (because they are flipped by the opponent's most recent move). The output of attention layers does not show the expected strong negative projection onto the MINE direction of most recent moves. Meanwhile, the attention layer output shows similarity to the board state of the previous move (board state after move 7, shown on the top left of Figure 10). We reproduce this figure for other moves in Appendix X.

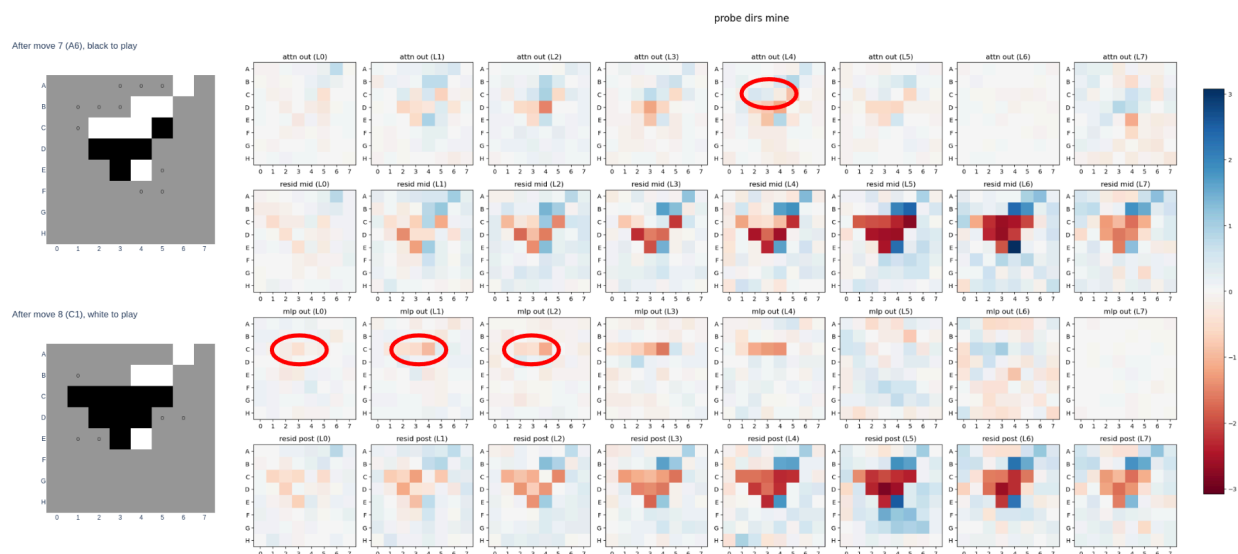


Fig 10: Left: Board states of a game after moves 7 and 8. Right: Heatmap of attributions of each component (attentions and mlps) on the “mine” probe direction after move 8. Red circled the tiles just flipped by move 8.

Quantitative evaluation across 500 games

We measure each block’s attribution to the just flipped squares across 500 games. We select the top k squares (top negative values) of each block attribution to the board states where k is determined by the actual number of just flipped squares for each move in each game. And we calculated the F1 score between top k squares based on board state attribution and the real k just flipped squares and plot every move and game in the below figure. We see L0 to L4 MLP’s board state attribution have a high F1 score with the just flipped squares.

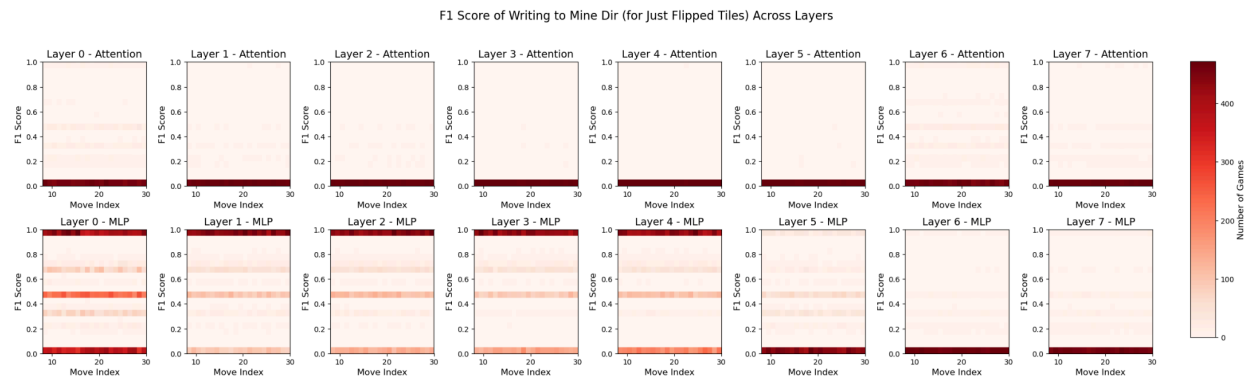


Fig 11: F1 score of top-k highest negative scores of \$tile projection = mlp_out @ mine_probe\$ as a predictor of flipped tiles. k is the number of tiles that are actually flipped in each single move and board state. The x-axis is the move indices of each game (move 8 to move 30), and the y-axis is the F1 score, with color representing the number of games that have this F1 score in each move.

Cross-token **Board State** Copying via Attention heads

The previous section described how MLPs compute the tiles flipped with the most recent move. The model must also reference tiles played in earlier turns to maintain a global board state. This is achieved via attention heads that transfer information across the token sequence. Fig 11 shows that attention heads do not write MINE / YOURS information based on FLIPS in the most recent move. Fig 10 indicates that attention heads do write prior board states.

Qualitative investigation of a single head

The following figure shows the average attributions of all Mine-heads, Yours-heads, Uniform-heads from layer 1, respectively, with the plates of a game in the first row. The attributions are calculated based on each dst token's contribution to the src token at move 8 on "mine" probe direction. It shows the Mine-heads writing tiles, including the tile played in that move, to mine.

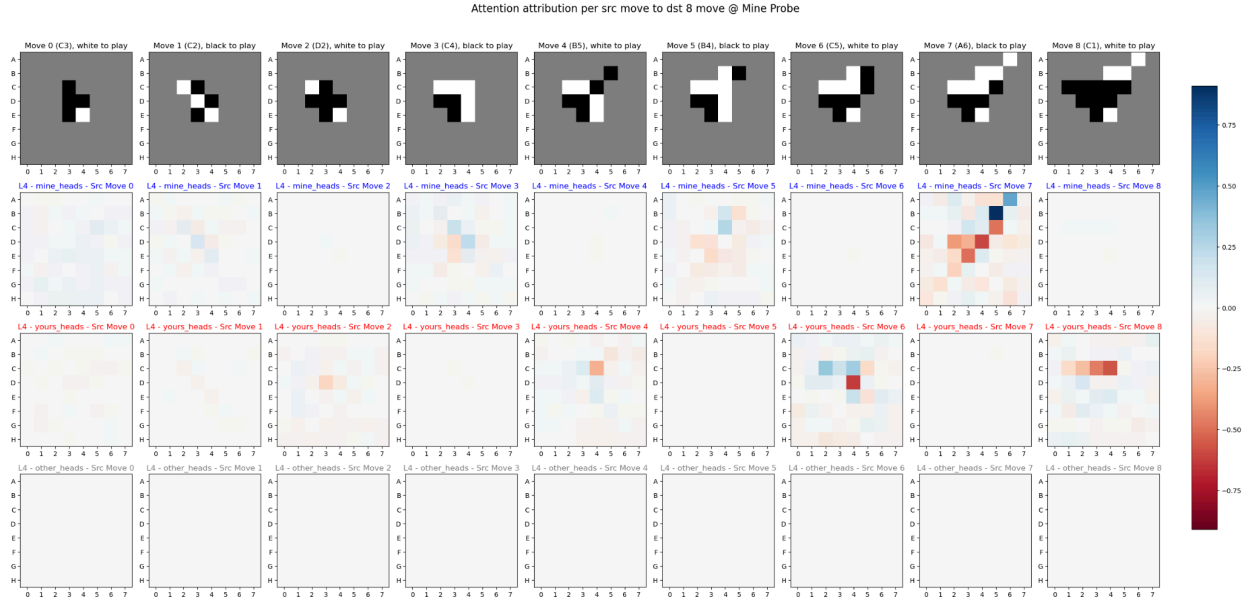
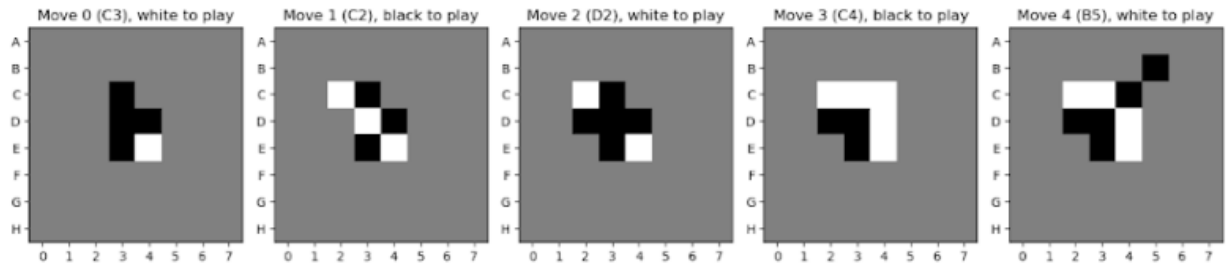


Fig 9: Average attributions for Layer-4 Mine-heads, Yours-heads, and Uniform-heads. The first row shows the board states of the game. Attributions measure each destination token's contribution to the source token at move 8 along the "mine" probe direction.

Board state → board state

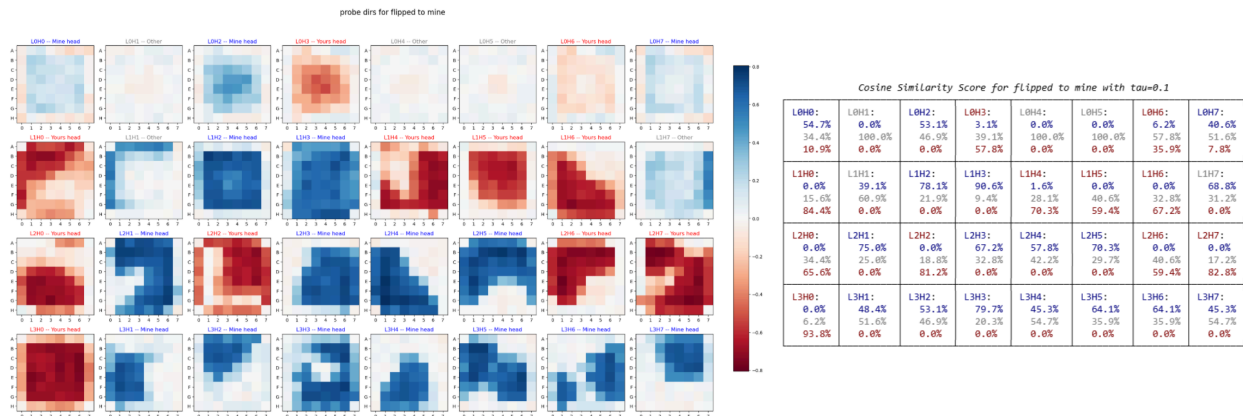
- aggregated view cancels out
- does it also read from just_flipped? → Next section.

[C3, C2, D2, C4, B5]



Weight-based analysis of the Attention OV matrices

Indeed, we now show the OV circuits read from previous flipped information, then write to the current board states. The OV matrix of an attention is a bijection, that reads from the residual stream and writes to the residual stream. Jmaar [cite] quantifies this as cosine similarity of "just flipped" probe @ OV @ "mine" probe and the attribution figure. In Fig 12 below, we reproduce how attention uses previous moves' flipped information then computes and contributes to current board states.



As described in the Background section, most attention heads either exclusively attend to MY moves or YOUR moves. We categorize them into MINE heads and YOURS heads accordingly. The Bijection of the OV matrices from just_flipped to mine aligns with the MINE vs YOURS heads categorization: MINE heads read from tiles that have been flipped by the current player in previous moves, and write flipped tiles to MINE direction accordingly. Analogously, YOURS heads read tiles that have been flipped in prior moves of the opponents, which are therefore yours.

Through OV (Output-Value) circuit analysis, we observe that "Mine" heads read "just played" information from previous tokens (played by me) and write it into the "mine" board state representation of the current token. This cross-token communication allows the model to "copy" the historical state forward, ensuring the current residual stream has access to the full board context.

Weight-based analysis: cosine similarity of “just played” probe @ OV @ “mine” probe.

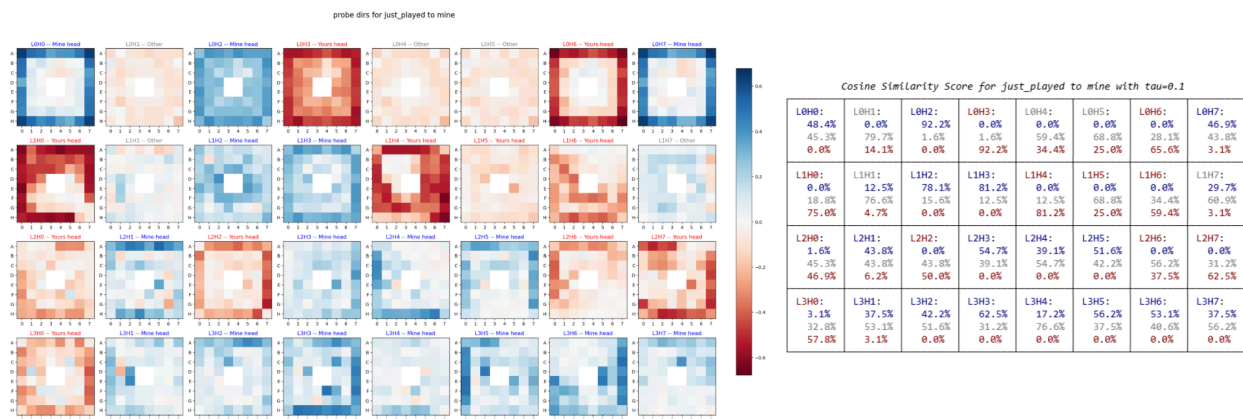


Fig 6: Left: cosine similarity of “just played” probe @ OV @ “mine” probe. Right: Fraction of relationships: positive relationships are defined as values greater than 0.1 (blue), negative relationships as values less than -0.1 (red), and all others are shown in grey. (Note, they don’t sum to 1 because the middle four squares are always pre-occupied before the start of the games.)

Board state inversion – OV Circuit

We next show the OV circuits read from board states information from previous tokens then flip and write to current board states.

Weight-based analysis: cosine similarity of “mine” probe @ OV @ “mine” probe.

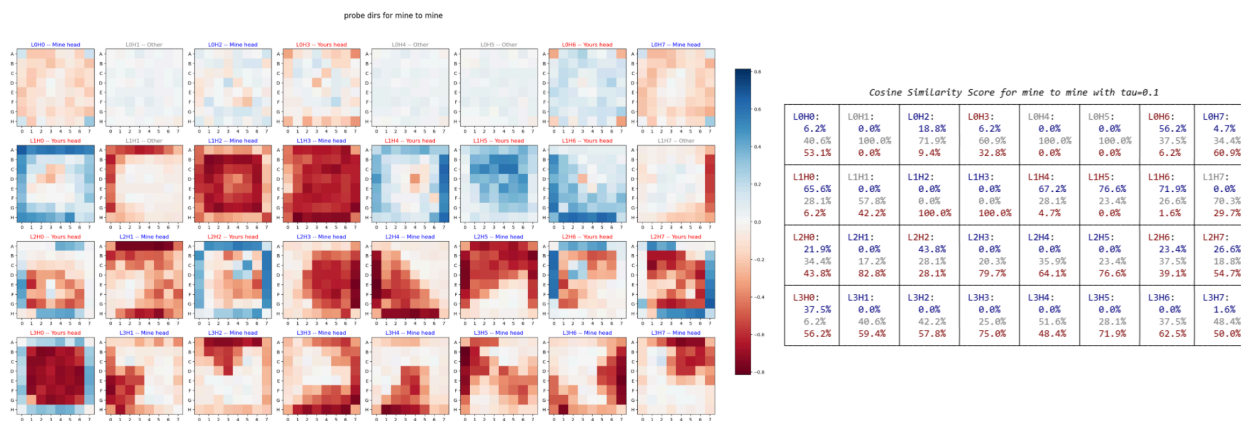


Fig 8: Left: cosine similarity of “mine” probe @ OV @ “mine” probe. Right: Fraction of relationships: positive relationships are defined as values greater than 0.1 (blue), negative relationships as values less than -0.1 (red), and all others are shown in grey.

This is counterintuitive. In the next section, we evaluate to which extent, the MINE, FLIPPED, and PLAYED information is causally relevant for model output. Maybe they suppress moves?

Causal intervention on OV circuits

Is “just flipped” and “just played” the *only* information determining the OV circuit output? We quantify this by zero-ablating the part of the input activations to the OV circuit, that lie orthogonal to the subspace spanned by the joint set of “just flipped” and “just played” directions for all squares. We measure the performance degradation in accuracy of predicting valid moves, and KL divergence to the original model output distribution without intervention. As a baseline we randomly select a subspace of same dimensionality, and ablate all orthogonal activation components.

These ablation studies targeting the value vectors of attention heads revealed that Layer 0 is particularly critical. When we restricted the model's communication to only "just flipped" and "just played" probe directions for all attention heads from Layer 1 onwards, the model maintained 93% accuracy. This intervention shows, that "just played" and "just flipped" directions are particularly relevant.

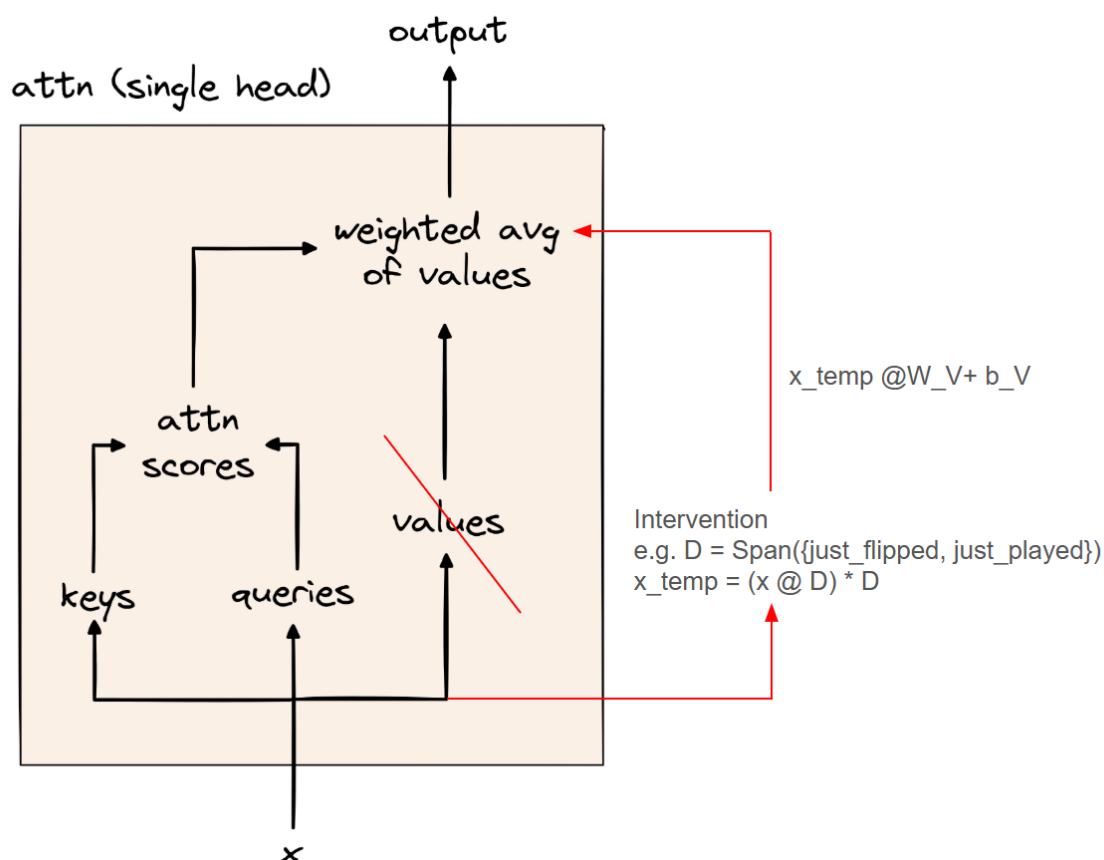


Fig 13: Intervention diagram

main takeaways

- L1-L7 this seems to be the only critical information, comparing accuracy with random accuracy.
- L0 has to carry some other heuristics? otherwise the performance shouldn't degrade

This suggests that the model's internal "world model" is remarkably efficient, discarding extraneous move data in favor of these two primary features. More results shown in (Table S1-3).

Attention Head Intervention Results
(Accuracy without intervention: 99.96%)

Intervention Layers	Accu.	KL	Accu. (Rand.)	KL (Rand.)	Accu. (Zero)	KL (Zero)
L0	77.10%	1.5368	57.87%	3.0199	71.22%	2.2077
L0 L1 L2 L3 L4 L5 L6 L7	62.43%	2.9379	36.70%	5.0697	28.30%	6.0091
L1 L2 L3 L4 L5	93.98%	0.1720	80.84%	0.5992	78.24%	0.5708
L1 L2 L3 L4 L5 L6 L7	93.72%	0.3457	78.69%	0.9711	77.07%	1.1587

Table 1: Accuracy and KL divergence for ablation studies on the value vectors of attention heads in designated layers. Results compare keeping the "flipped" and "just played" directions and zero ablating other directions, keeping random (same number of) directions, and setting the value vectors to zero.

Discussion of board state computation.

What would a minimal transformer be to implement these algorithms?

Valid move Lookup

The final phase of the model's computation involves translating the reconstructed board state into move predictions. To decode this, we automated the discovery of "rule-based" neurons using regression decision trees to map board features to neuron activations.

Below shows some main figures in our previous workshop paper:

<https://arxiv.org/abs/2511.00059>

We automate the process of finding rule-based neurons in OthelloGPT by training regression decision trees that map board state features to neuron activations, and then extract decision paths where neurons are highly active to convert neurons into human-readable logical forms (Figure 1). The result is a disjunctive normal form (DNF; OR-of-ANDs) description for each neuron, where each AND corresponds to a high activation decision path of the neuron's decision tree. We may then specify a rule-base game query (an AND of features), and automatically surface the implementing neurons by evaluating if their DNF evaluates to True when given the query.

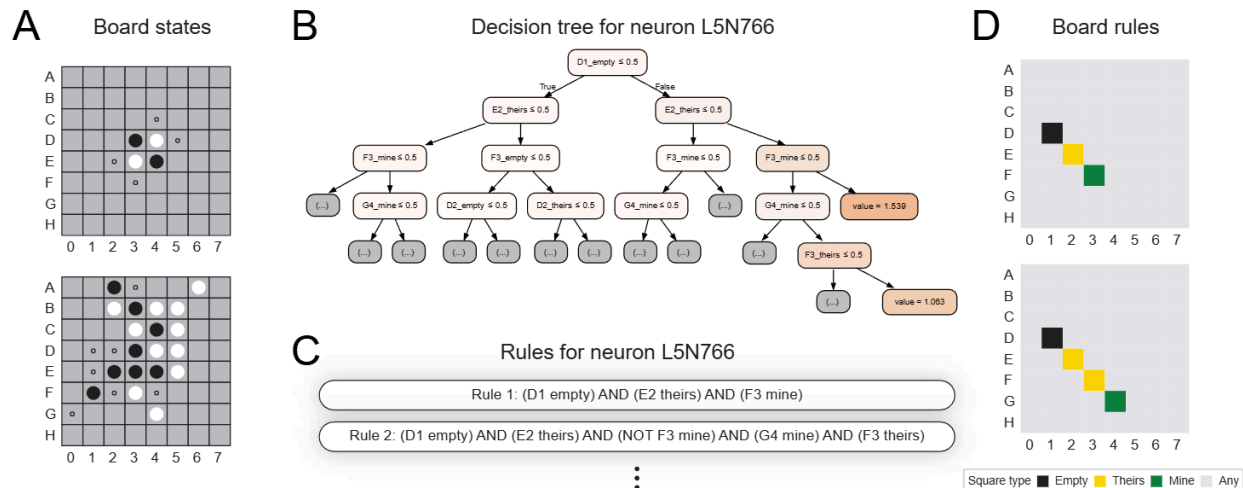


Fig 14: Overview of neuron interpretation pipeline. Board state features are used as training data for decision trees to predict neuron activations ($\text{neuron}\{A\}$, $\text{neuron}\{B\}$). Note: small circles in $\text{neuron}\{A\}$ and $\text{neuron}\{B\}$ are all possible legal moves. High activation decision paths are then extracted to obtain logical rules for each neuron ($\text{neuron}\{C\}$, $\text{neuron}\{D\}$). We provide an interactive visualization of decision trees in this <https://colab.research.google.com/drive/1kKLj9c3eIB0yjJoBZnkHqFI3ZWYgP6zw>

We perform an intervention experiment along these lines for each of the 60 squares that can be played in OthelloGPT. For each square, we specify an intervention condition and a control condition, both of which are distinct 3 square patterns that make the given square legal. A visual depiction is shown in (Figure 5 A, B).

We include natural metrics such as probability difference and logit difference. While another natural metric to include would be the change in legal move accuracy (defined as inaccurate if and only if an illegal square is in the top K logits, where K is the number of legal moves in the given position), OthelloGPT's prediction of a uniform distribution over legal moves complicates this metric. In particular, since each legal move receives probability $1/K$ (where K is the number of legal moves), and each illegal move receives probability 0, it is difficult to completely move the intervention legal square out of the top K. The situation is visually depicted in Figure 5 C, D (observe the shading for square H3). Hence, we instead include the following three metrics: dropping the intervention square probability below 1%, 5%, and 10% of its original probability.

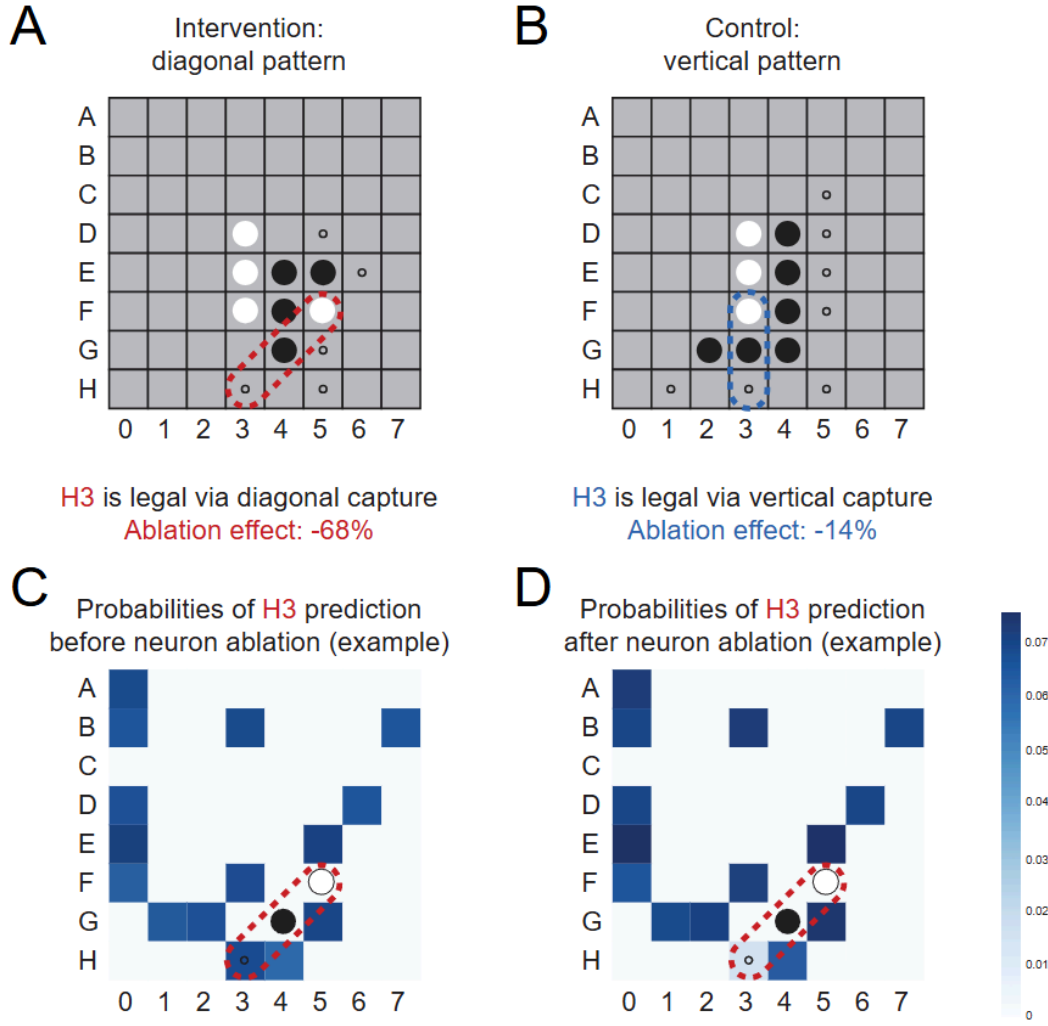


Fig 15: Fine-grained intervention. **A**: Intervention pattern example: valid move via diagonal pattern. **B**: Control pattern example: valid move via diagonal pattern. Note: small circles in **A** and **B** are all possible legal moves. **C**, **D**: Output Probabilities of OthelloGPT output on a game example where H3 is legal via diagonal pattern. **C**: Before intervention. **D**: After intervention, the probability for the intervened square H3 drops to in between the legal and illegal square probabilities.

Averaging results over all 60 squares, we find that the legal square probability drops nearly twice as much in the intervention conditions as in the control conditions (Table 1). For the below 1% of its original probability metric, we find ~4x greater effect in the intervention than control conditions.

We found that intervention results for the 12 squares adjacent to the middle 2x2 region were subpar, suggesting that there is greater shared behavior among such squares. Hence, we perform a variant where we average only over the 48 squares outside the middle 4x4. As shown in Table 1, we find substantial improvement in our metrics, most notably with a ~10x greater

effect in the intervention than control conditions for the below 1% of original probability metric (0.077 vs. 0.0076). We interpret these results as evidence that the neurons causally implement the behaviors their decision trees describe, but note that the incomplete effect on model behavior suggests that there are other prediction mechanisms that we are missing.

Metric	All 60 Squares		48 Squares (excl. middle 4x4)	
	Intervention	Control	Intervention	Control
Logit Difference	1.612	0.723	1.854	0.682
Probability Difference	0.057	0.031	0.062	0.031
Clean Accuracy	0.99996	0.99999	0.99995	0.99998
Corrupted Accuracy	0.994	0.997	0.998	0.999
Accuracy Difference	0.0059	0.0030	0.0016	0.00065
Below 1%	0.058	0.015	0.077	0.0076
Below 5%	0.191	0.062	0.239	0.040
Below 10%	0.287	0.094	0.344	0.079

Table 2: Intervention vs. Control Condition Results

Discussion

- Quantify “how much of the true model computation we are explaining with these four mechanisms”.

Related work

Another line of work [6] extended linear probing across layers and claimed that OthelloGPT learned a diverse set of localized heuristics. [4] and [5] trained Sparse Autoencoders (SAEs) on later layers of OthelloGPT to study its internal representations. The SAEs were able to accurately reconstruct game boards and capture several game-specific features. However, they still underperformed compared to linear probes.

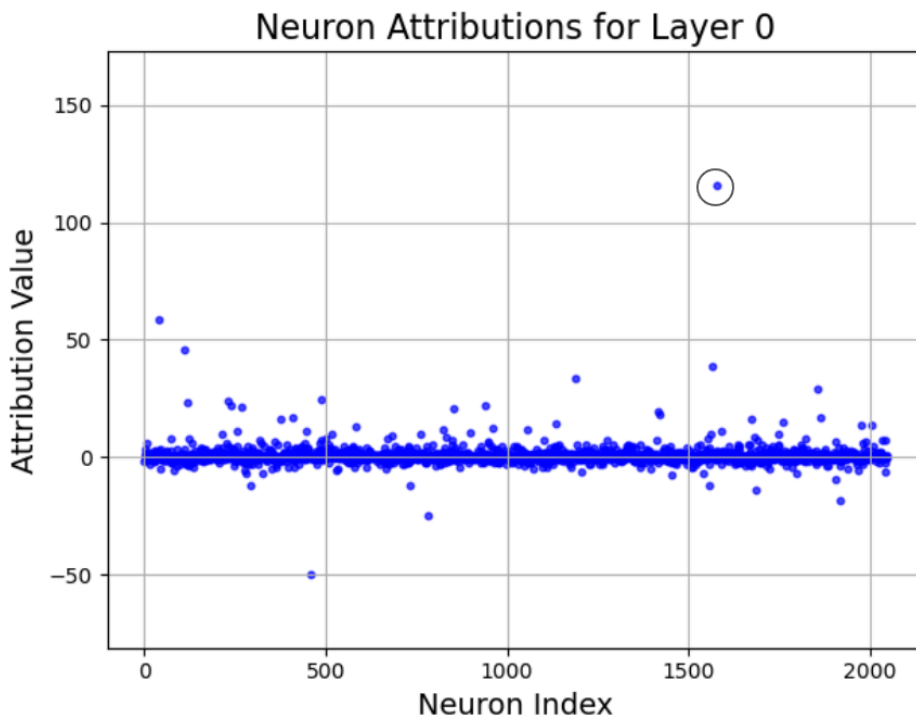
Authors [6] further identified specific neurons that encode mechanistic rules and perform logical computations, such as detecting whether a square is blank. Others acted as Board Pattern Neurons for legal move detection or as Clock Neurons tracking game phase. These findings suggest that the model distributes its reasoning across specialized components, forming a kind of “bag of heuristics.”

[7] proposed the Previous Color Circuit, a mechanism by which the model copies over board state through a weighted average of prior moves and flips in the Mine and Yours attention heads. The study also identified mono-semantic neurons involved in tile flipping, but attempts to isolate a flipping circuit through ablation were unsuccessful. This suggests that circuit-level interactions, rather than individual neurons performing interpretable rules, may underlie some of the model’s behavior.

Acknowledgements

Work done as part of the Algoverse Program. We thank our team members of the previous project, Aditya Singh, Srujananjali Medicherla, Adam Karvonen, and mentors Callum McDougall, Kevin Zhu, and Sunishchal Dev. Andy Arditì gave helpful feedback on the narrative.

Appendix



(Rank 1: L0N1578) MLP Weights Projection for square C2

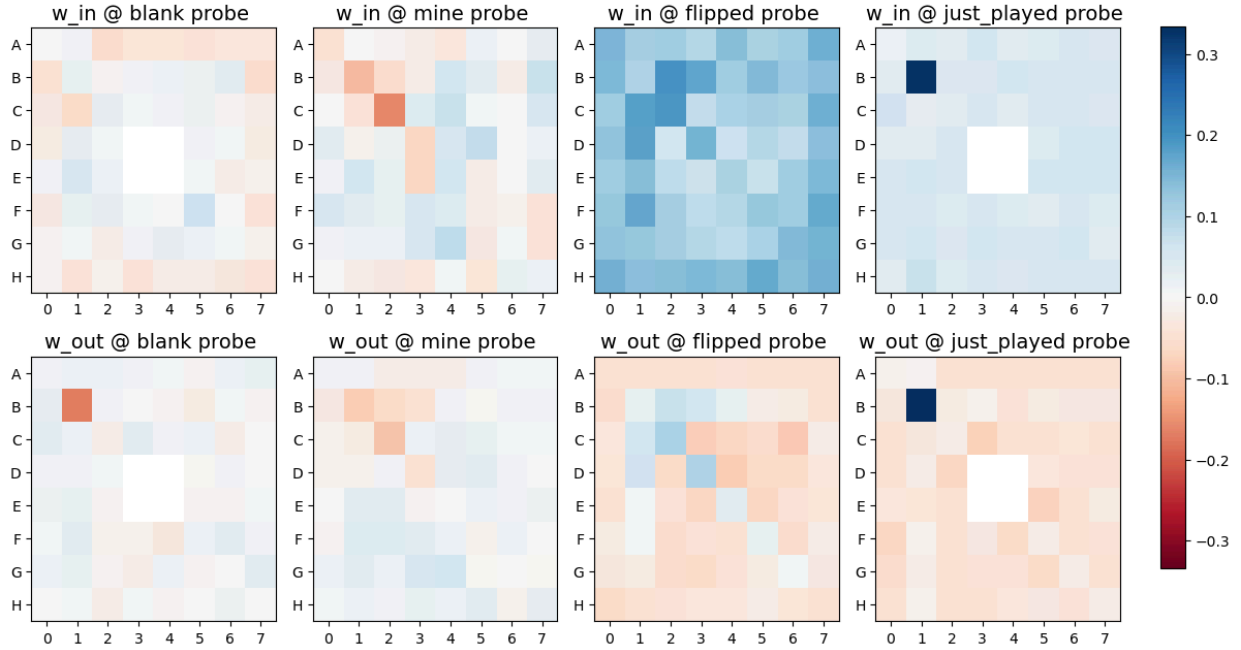


Fig S1: Top: Attribution score for each L0 neuron's contribution to C2 flipped probe direction with all moves where C2 flipped over 500 games. Black circled neuron L0N1578. Bottom: cosine similarity of W_{in} and W_{out} with “blank”, “mine”, “flipped”, “just played” probe directions, respectively.

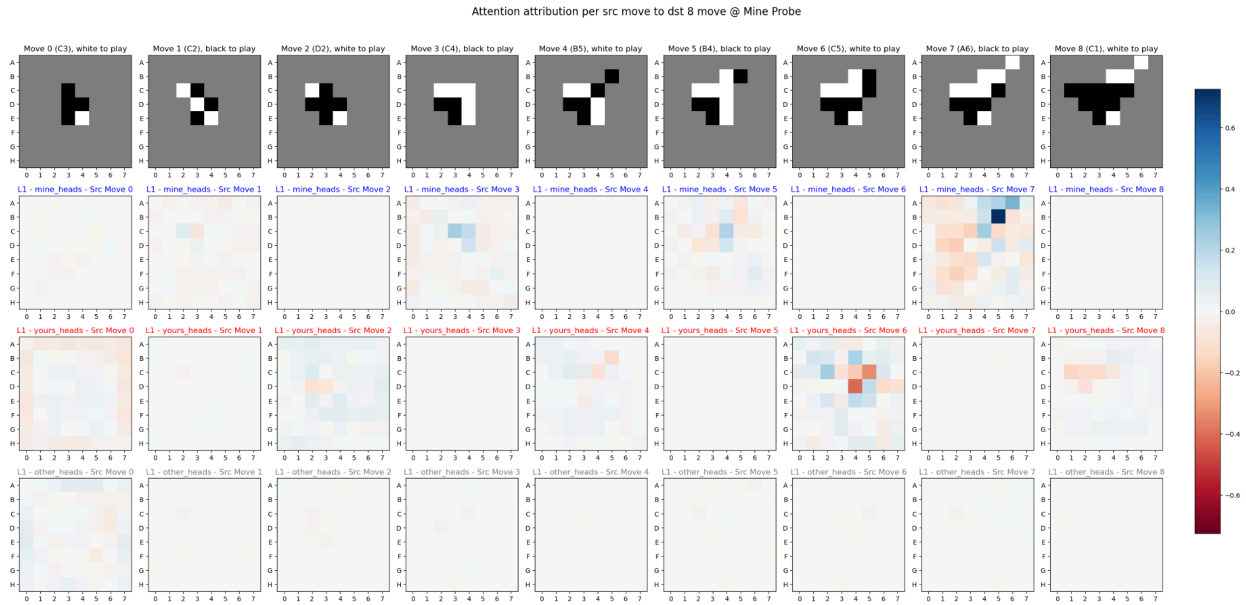


Fig 7: Average attributions for Layer-1 Mine-heads, Yours-heads, and Uniform-heads. The first row shows the board states of the game. Attributions measure each destination token's contribution to the source token at move 8 along the “mine” probe direction.

The second table released the model's communication to only "flipped", "just played", and "mine" probes (Table S1). It shows the attention heads also use "mine" probes from previous moves to compute board states. However, the attribution of this part is comparably small but non-negligible.

Attention Head Intervention Results (Accuracy without intervention: 99.96%)						
Intervention Layers	Accu.	KL	Accu. (Rand.)	KL (Rand.)	Accu. (Zero)	KL (Zero)
L0	75.01%	1.7037	60.13%	2.7260	71.22%	2.2077
L0 L1 L2 L3 L4 L5 L6 L7	61.41%	2.8164	41.81%	3.7337	28.30%	6.0091
L1 L2 L3 L4 L5	94.92%	0.1610	85.64%	0.4068	78.24%	0.5708
L1 L2 L3 L4 L5 L6 L7	94.94%	0.2722	82.97%	0.5865	77.07%	1.1587

Table S1: Accuracy and KL divergence for ablation studies on the value vectors of attention heads in designated layers. Results compare keeping the "flipped", "just played" and "mine" directions, keeping random (same number of) directions, and setting the value vectors to zero.

We also performed supplementary experiments where we remove the corresponding directions from the value vectors of attention heads and keep all the other directions (Table S2-3). The accuracy drops significantly compared to controls and shows a similar pattern.

Attention Head Intervention Results (Accuracy without intervention: 99.96%)						
Intervention Layers	Accu.	KL	Accu. (Rand.)	KL (Rand.)	Accu. (Zero)	KL (Zero)
L0	62.52%	2.4095	79.85%	1.3098	99.96%	0.0000
L0 L1 L2 L3 L4 L5 L6 L7	45.37%	3.1132	67.80%	1.8043	99.96%	-0.0000
L1 L2 L3 L4 L5	91.64%	0.2415	98.13%	0.0708	99.96%	0.0000
L1 L2 L3 L4 L5 L6 L7	89.23%	0.2771	97.43%	0.0923	99.96%	0.0000

Table S2: Accuracy and KL divergence for ablation studies on the value vectors of attention heads in designated layers. Results compare removing the "flipped", "just played" directions, removing random (same number of) directions, and setting the value vectors to zero.

Attention Head Intervention Results (Accuracy without intervention: 99.96%)						
Intervention Layers	Accu.	KL	Accu. (Rand.)	KL (Rand.)	Accu. (Zero)	KL (Zero)
L0	61.34%	2.7085	64.24%	2.3760	99.96%	0.0000
L0 L1 L2 L3 L4 L5 L6 L7	38.24%	3.6215	50.17%	3.1628	99.96%	-0.0000
L1 L2 L3 L4 L5	82.33%	0.4775	93.67%	0.1969	99.96%	0.0000
L1 L2 L3 L4 L5 L6 L7	77.82%	0.5508	92.25%	0.2726	99.96%	0.0000

Table S3: Accuracy and KL divergence for ablation studies on the value vectors of attention heads in designated layers. Results compare removing the "flipped", "just played" and "mine" directions, removing random (same number of) directions, and setting the value vectors to zero.

Reference

- [1] Li, Kenneth, et al. Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. arXiv:2210.13382, arXiv, 26 Jun. 2024. arXiv.org, <https://doi.org/10.48550/arXiv.2210.13382>.
- [2] Nanda, Neel, et al. Emergent Linear Representations in World Models of Self-Supervised Sequence Models. arXiv:2309.00941, arXiv, 7 Sep. 2023. arXiv.org, <https://doi.org/10.48550/arXiv.2309.00941>.
- [3] Hazineh, Dean S., et al. Linear Latent World Models in Simple Transformers: A Case Study on Othello-GPT. arXiv:2310.07582, arXiv, 12 Oct. 2023. arXiv.org, <https://doi.org/10.48550/arXiv.2310.07582>.
- [4] Karvonen, Adam, et al. Measuring Progress in Dictionary Learning for Language Model Interpretability with Board Game Models. arXiv:2408.00113, arXiv, 30 Oct. 2024. arXiv.org, <https://doi.org/10.48550/arXiv.2408.00113>.
- [5] Robert_AIZI. Research Report: Sparse Autoencoders Find Only 9/180 Board State Features in OthelloGPT. <https://www.lesswrong.com/posts/BduCMgmjJnCtc7jKc/research-report-sparse-autoencoders-find-only-9-180-board>.
- [6] Lin, Jennifer, et al. OthelloGPT Learned a Bag of Heuristics. <https://www.lesswrong.com/posts/gcpNuEZnxAPayaKBY/othelloGPT-learned-a-bag-of-heuristics-1>.

[7] JMaar. Exploring How OthelloGPT Computes Its World Model.

<https://www.lesswrong.com/posts/wezSznWnsMhpRF2QH/exploring-how-othelloGPT-computes-its-world-model>.

[8] Ameisen, et al., "Circuit Tracing: Revealing Computational Graphs in Language Models",

Transformer Circuits, 2025. <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>

Idea

- 100X100 to see move depth limitations